```
                    "Bearer error=\"insufficient_scope\","
+    #C
                              "scope=\"" + requiredScope +
"\"");    #C
            halt(403);    #C
        }
    };
}
```

#A If the HTTP method does not match then ignore this rule.
#B If the token is unscoped then allow all operations.
#C If the token scope doesn't contain the required scope then return a 403 Forbidden response.

You can now use this method to enforce which scope is required to perform certain operations, as shown in listing 7.3. Deciding what scopes should be used by your API, and exactly which scope should be required for which operations is a complex topic, which is discussed in more detail in the next section. For this example, you can use fine-grained scopes corresponding to each API operation: `create_space`, `post_message`, and so on. To avoid privilege escalation, you should require a specific scope to call the login endpoint, because this can be used to obtain a token with any scope, effectively bypassing the scope checks.[2] On the other hand, revoking a token by calling the logout endpoint should not require any scope. Open the Main.java file in your editor and add scope checks using the `tokenController.requireScope` method as shown in listing 7.3.

Listing 7.3 Enforcing scopes for operations